

Technical Case Study: Real-Time Patient Flow & Bed Occupancy Analytics

Engineer: Abhiram Chinnam **Domain:** Healthcare / Real-Time Analytics **Tech Stack:** Azure Event Hubs, Databricks (PySpark), ADLS Gen2, Synapse Analytics, ADF.

1. Executive Summary

The Problem: A multi-hospital network ("Midwest Health Alliance") faced operational inefficiencies due to a lack of visibility into real-time patient flow. Bed occupancy data was only available via nightly batch reports, leading to overcrowding in Emergency Departments (ED) and inefficient staff allocation.

The Solution: I engineered an enterprise-grade **Real-Time Data Streaming Platform**. The system ingests live patient event data, processes it via a **Medallion Architecture**, and serves up-to-the-minute dashboards on bed availability and wait times.

Key Impact:

- Reduced data latency from **24 hours to <2 minutes**.
- Enabled historical tracking of patient movements using **SCD Type 2** logic.
- Automated the entire workflow using **Azure Data Factory**, reducing manual maintenance.

2. System Architecture & Security

2.1 The "Decoupled" Ingestion Strategy

I utilized **Azure Event Hubs** as the ingestion buffer rather than writing directly to the database.

- **Why?** This decouples the source (Patient Simulator) from the processing engine (Databricks). If Databricks goes down for maintenance, Event Hubs retains the data (retention period), ensuring **zero data loss**.
- **Throughput:** Capable of handling millions of events/second, future-proofing the hospital for IoT expansion.

2.2 "Zero Trust" Security Implementation

A critical requirement was securing patient data (HIPAA compliance).

- **No Hard-Coded Secrets:** I strictly avoided pasting connection strings in notebooks.
- **Azure Key Vault:** All keys (Event Hub Connection, Storage Keys) are stored in Key Vault.
- **Databricks Secret Scopes:** I utilized `dbutils.secrets.get(scope="hospitalanalysisvaultscope")` to inject credentials securely at runtime.

3. The Data Pipeline (Medallion Architecture)

I implemented a "Lakehouse" pattern where data moves through three quality gates stored in ADLS Gen2.

Step 1: Bronze Layer (Raw Ingestion)

- **Goal:** Capture data exactly as it arrives to ensure a reliable audit trail.
- **Logic:**
 - Ingested binary data from Event Hubs.
 - Converted binary payload to String.
 - Wrote to ADLS Gen2 as a **Delta Table** (`append` mode).
- **Fault Tolerance:** Enabled **Checkpointing**. This ensures that if the cluster crashes, the stream restarts from the exact last processed offset (Exactly-Once Processing).

Step 2: Silver Layer (Cleansing & Evolution)

- **Goal:** Turn raw JSON into structured, typed data.
- **Challenge:** The source system occasionally adds new fields (e.g., `insurance_id`) without warning.
- **Solution:** I enabled **Schema Evolution** using `.option("mergeSchema", "true")`. This prevents the pipeline from breaking when data drifts.
- **Transformation:** Parsed JSON, cast timestamps, and handled NULL values.

Step 3: Gold Layer (Star Schema & History)

- **Goal:** Business-ready data for reporting.
- **Modeling:** Designed a **Star Schema** with:
 - **Fact Table:** `fact_patient_flow` (Admissions, Discharges).
 - **Dimension Tables:** `dim_department`, `dim_patient`.
- **Advanced Logic (SCD Type 2):**
 - I implemented **Slowly Changing Dimensions (Type 2)** to track patient history.

- Example: If a patient moves from "ICU" to "General Ward," the system closes the old record (`is_current=False`) and creates a new active record. This allows the hospital to audit *exactly* where a patient was at any point in time.

4. Orchestration & Serving

Automation (Azure Data Factory)

- Created a scheduled trigger (`Goldtrigger`) that runs the pipeline every 15 minutes.
- Used **Linked Services** to securely manage the connection between ADF and Databricks.

Serving (Synapse Analytics)

- Instead of expensive direct-writes, I used the **External Table** pattern.
- Databricks writes optimized Delta files to the Gold Container.
- **Synapse Serverless SQL** reads these files directly, allowing Power BI to query massive datasets without requiring a dedicated warehouse cluster running 24/7